独立按键控制LED亮灭 / 控制LED状态 / 控制LED显示二进制 / 控制LED移位

对应江协科技《51单片机入门教程-2020版》BV1Mb411e7re 第3章 · 范围 = P7 [3-1]独立按键控制LED亮灭、P8 [3-2]独立按键控制LED状态、P9 [3-3]独立按键控制LED显示二进制、P10 [3-4]独立按键控制LED移位 · 本章目标：学会"读 IO 口寄存器"检测按键；理解按键一脚 GND、按下读低电平的接法原理；掌握软件延时消抖与"检测松手"标准模板；用 if/else 做按下亮灭、松手取反；用 unsigned char 中间变量做 8 灯二进制递增；用 0x01<<n 做 K1/K2 双向移位；补 C51 的 sbit/sfr、位运算、函数原型、局部/全局变量 · 不覆盖：4×4 矩阵键盘、定时器/中断等后续章节内容

本节范围

本块是 51 单片机第一次做**"输入 → 输出"**闭环：上一章是 CPU **写** P2 寄存器点灯，这一章反过来，CPU **读** P3 口寄存器看按键被按下没有。主线只有一条——**按键一脚接 GND、一脚接 IO，松开时 IO 被内部上拉拉成高电平(1)，按下时接通 GND 变低电平(0)；程序里 `if(P3_1==0)` 读到 0 就是被按下了**。四集依次加码：按下亮松开关 → 松手翻转一次 → 按一次加一个二进制数 → K1/K2 双向移位。

> **学习提示：**这一章不要背四段代码，要背同一个"按键标准模板"—— `if(读到按下){ Delay(20); while(按键仍按下); Delay(20); 动作; }` 。四集的区别只在最后那句"动作"：亮灭、取反、自增送 P2、移位送 P2。另外两个硬记点：**板子 K1/K2 丝印与 IO 反接（K1=P3.1、K2=P3.0）**，以及**要用位名 P2_0/P3_1 必须 include regx52.h（reg52.h 没有 sbit）**。



实物图 · 普中 HC6800-ES V2.0 开发板整板（课程同款）：左下角"独立键盘"丝印处即 K1–K4 四个独立按键，右侧为沿用第2章的 D1–D8 排灯 · 图源：网络搜索

核心知识点

内容	条件 / 说明	易错
独立按键结构	就是金属弹片电子开关，按下接通、松开断开；4 脚直插轻触开关，同侧两脚始终导通，按下时 4 脚全通（P7 01:48–04:36）	4 脚不是两两独立，同侧两脚是同一点

检测原理	按键公共一脚接 **GND(电源负极)**，另一脚引出连到 MCU 的 IO；寄存器除了"写值送 IO"，还能**读回 IO 当前电平**——这就是读按键（P7 04:36–05:50）	别只会写 P2，P3 是"读"进来判断的
按下/松开电平	51 上电所有 IO 口默认高电平；松开时 IO 被内部上拉拉高为 **1**，按下时接通 GND 变 **0**（P7 05:16、P9 06:11）	按下读回的是**低电平 0**，不是 1
按键 ↔ IO 对应	板子丝印反了：**K1=P3.1(RXD)、K2=P3.0(TXD)、K3=P3.2、K4=P3.3**；按常理 K1 应是 P3.0，但 RXD/TXD 位置画反了（P7 12:10–13:10、P10 11:49）	丝印 K1 不是 P3.0！写成 P3.0 会按错键
sfr / sbit	`sfr` 定义 8 位整体寄存器，`sbit` 定义其中某一位、可单独读写；位操作只给 0/1 即可（非 0 即真）（P7 08:18–10:20）	想只改一位不动其它位，用 sbit 而不是整个 P2 赋值
头文件区别	`reg52.h` 里只有 sfr 没有 sbit；`regx52.h` 同时有 sfr 和 sbit；要用 P2_0、P3_1 这种位名必须 include **regx52.h**（或自己用 sbit 声明）（P7 09:23–11:16）	用 reg52.h 写 P2_0 会报"P2_0 未定义"
位操作点灯	`P2_0 = 0;` 单独点亮 D1 不影响其它 LED，`P2_0 = 1;` 熄灭；D1 接 P2.0，沿用共阳低电平点亮（P7 07:44、11:16）	—
if 读按键 / while(1)	第一版：`while(1){ if(P3_1==0){P2_0=0;}else{P2_0=1;} }`，按下亮松开灭；**if/else 必须放进 while(1) 里**（P7 13:10–15:59）	if 放外面上电只判断一次，按复位才再跑一遍
按键抖动	机械触点闭合/断开瞬间金属片弹性抖动几下，**抖动时间约 5~10ms**；单片机太快会把一次按下识别成多次（P8 00:00–01:10）	不做消抖会出现"按一次跳好几下"
软件消抖	检测到按下后 `Delay(20)` 跳过按下抖动，等松手后再 `Delay(20)` 跳过松手抖动；硬件消抖要加触发器电路，本课用软件法（P8 02:11、10:06–11:20）	消抖要按、松两头都做，只做一头仍会抖
等松手空语句	消抖后 `while(P3_1==0);` 空转直到松手（空循环体直接写分号，等价一对空花括号），再做动作实现"松手才动作"（P8 09:30–10:43）	分号不能漏：`while(P3_1==0);` 的 ; 是语句本体
带参 Delay	把 1ms 空转函数改造成 `Delay(unsigned int xms){ while(xms){ 1ms空转; xms--; } }`；	传超过 65535 会溢出绕回 0；x-- 后置用完再减

	unsigned int 范围 0~65535，传参别超（P8 04:51–06:15）	
状态翻转	"按一下亮、再按一下灭"：`P2_0 = ~P2_0;`，波浪线对 sbit 一位直接取反（P8 11:20）	—
不能直接操作 P2	P2 上电默认 **0xFF(全高)**；直接 `P2++` 溢出变 0x00、`P2=~P2` 又回 0xFF，永远卡死在全亮/全灭两个极端（P9 04:32–06:11）	二进制递增/移位不能直接对 P2 寄存器自增取反
中间变量中转	定义 `unsigned char LED_number=0;`，对变量自增/移位，再 `P2=~LED_number;` 取反送 P2；unsigned char 8 位、范围 0~255，正好匹配寄存器位宽（P9 07:28–08:52）	变量在 = 右边是读、左边是写；P2=~LED_number 不改 LED_number
标识符命名	变量名只能由**字母、数字、下划线**组成，**不能以数字开头**，不能加其它特殊字符（P9 07:56）	如 1key、key# 都是非法变量名
位运算	单片机高频：`<<` 左移(高位移出低位补0)、`>>` 右移；`&` 全1出1、`|` 有1出1、`^` 不同出1、`~` 逐位取反（P7 25:48–32:01）	按位与/或是两位逐位算；逻辑 &&/|| 才是两条件整体算
函数原型声明	子函数定义在 main 后面时，必须在 main 前**声明函数原型**（函数头末尾加分号），否则报"missing function-prototype"；报错一连串时双击第一个错误定位（P10 01:38–03:11）	分号不能省；P 必须大写（P3，小写 p 是另一个标识符，大小写敏感）
局部 vs 全局变量	局部变量写在函数体里、仅本函数用，不赋初值则初值随机；全局变量写在函数外、所有函数共用，**上电默认初值 0**（P10 04:14）	要在多次按键间累计移位位数，必须用全局变量
双向移位	用 `P2 = ~(0x01 << LED_number);`：0x01 左移 n 位再整体取反（低电平点亮）；K1=P3.1 左移 LED_number++（到 8 归零），K2=P3.0 右移 LED_number--（到 0 跳 7）（P10 06:22–13:55）	~( ) 的括号不能省（运算符优先级）；改 Create HEX 后必须 Rebuild
上电初始化	开头 `P2 = ~0x01;`（=11111110，仅 D1 亮），避免上电全灭、按第一下直接跳到 D2（P10 10:30）	少了这句，上电按第一下灯就从 D2 开始

> **接法决定逻辑（一句话记牢）：**按键一脚固定接 GND，另一脚接的 IO 平时被内部上拉拉成高电平 **1**；手指按下把 IO 和 GND 短接，IO 才掉到 **0**。所以程序是 `if(按键 == 0)` 判断"被按下"——读到 0 不是出错，正是按下的信号。

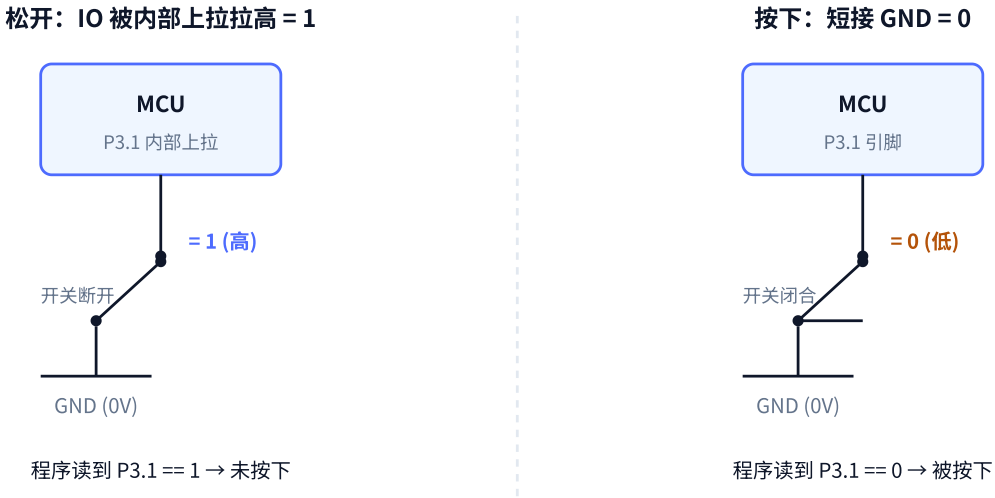


图 ①　独立按键原理：一脚固定 GND、一脚接 IO；松开时 IO 被内部上拉拉成高电平 1，按下时与 GND 短接掉到 0，程序 `if(P3_1==0)` 即检测到按下（P7 04:36–05:50）。

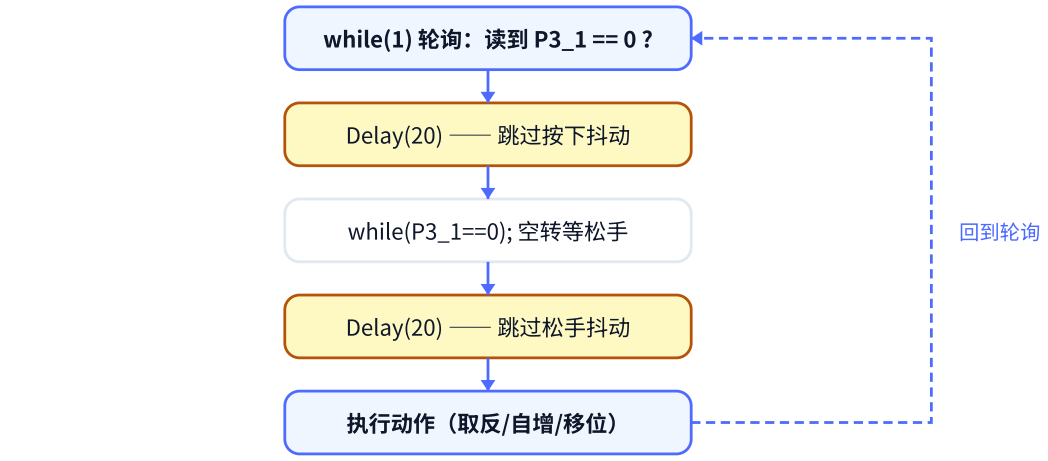


实物图 · 4 脚直插轻触开关：同侧两脚始终导通，按下时 4 脚全部接通；开发板左下角"独立键盘"丝印处即 4 个这种按键 · 图源：网络搜索

易错点

- **丝印反接：**K1 实际是 **P3.1(RXD)**、K2 是 **P3.0(TXD)**，不是直觉的 K1=P3.0；写错就按错键（P7 12:10）。
- **头文件：**要用 P2_0/P3_1 位名必须 include **regx52.h**；用 reg52.h 会报"P2_0 未定义"（P7 09:23）。
- **== vs =：**判断相等写 `==`（两个等号）；if 里写一个 `=` 是把 0 赋给 P3_1，逻辑完全错（P7 22:42、P8 07:00）。
- **必须放 while(1)：**if/else 放在 while(1) 外面，按键只在上电/复位时生效一次（P7 14:35）。
- **消抖两头做：**检测到按下 Delay(20)、等松手后再 Delay(20)；抖动约 5~10ms，不跳过会一次按识别成多次（P8 00:00、10:06）。
- **全角符号：**中文全角分号/符号编译报 "unprintable character"，必须切回英文输入法（P8 11:20）。

- **传参范围：**Delay 的 unsigned int 实参别超 65535，溢出会绕回 0（P8 04:51）。
- **别直接操作 P2：**P2 上电 0xFF，直接 P2++/P2=~P2 会卡在全亮/全灭两极，必须用 unsigned char 中间变量中转（P9 06:11）。
- **函数原型：**函数定义在 main 后面时，前面要加分号声明原型；报错一串就双击第一个（P10 01:38、02:24）。
- **括号与大小写：**`~(0x01<<n)` 的括号不能省（优先级）；P 必须大写（P3，不是 p3）（P10 03:11、08:50）。
- **Create HEX 后 Rebuild：**勾了 Options→Output 的 Create HEX 后必须重新编译，新 .hex 才出现（P10 10:30）。



机械抖动约 5~10ms：按下、松开两头各用 Delay(20) 跳过，中间 while 等松手，动作只在松手那一刻做一次

图 ②　按键标准消抖模板：检测按下 → Delay(20) → 等松手 → Delay(20) → 执行动作，松手才动作；这是 P8/P9/P10 共用的骨架（P8 02:11–11:20）。

常见题型

题型	做法要点
由接法判断按下电平	按键一脚 GND、一脚 IO；松开内部上拉=高电平 1，按下接通 GND=低电平 0；`if(按键==0)` 即按下
按下亮 / 松开灭	while(1){ if(P3_1==0){P2_0=0;}else{P2_0=1;} }；if 必须在 while(1) 内
按一下翻转一次	消抖模板后 `P2_0 = ~P2_0;`；先 Delay(20)、while(P3_1==0); 等松手、再 Delay(20)
8 灯二进制递增	定义 unsigned char LED_number=0；消抖模板后 `LED_number++;` `P2=~LED_number;`，不能直接 P2++

K1/K2 双向移位	`P2=~(0x01<<LED_number);`；K1++(≥8 归零)、K2--(==0 跳 7)，K1/K2 用两个独立 if；上电 P2=~0x01
带参延时改造	1ms 空转函数外包 `while(xms){ …; xms--; }`，传参即延时毫秒数，范围 0~65535
编译报错排查	P2_0 未定义→换 regx52.h；unprintable→切英文输入法；missing prototype→函数前加分号；P 大小写敏感

例题速览

下面四段 C 代码均为**自写示例（非原课程源码）**，按标准 Keil C51 / 8051 语法组织；延时函数体由 STC-ISP 计算器按 12.000MHz / Y1 生成，讲义只写骨架不抄内部循环次数。

例 1（P7 按下亮、松开灭） K1 按下时 D1 亮，松开时 D1 灭。

- **思路：**K1=P3.1（板子反接），按下读回低电平 0；D1=P2.0 低电平点亮。if/else 包在 while(1) 里轮询。
- **做法：**

```
#include <regx52.h>   /* 自写示例（非原课程源码）：regx52.h 才有 P2_0/P3_1 位名 */

void main()
{
    while(1)
    {
        if(P3_1 == 0)   /* K1 按下接地，读回低电平 */
        {
            P2_0 = 0;   /* 点亮 D1 */
        }
        else
        {
            P2_0 = 1;   /* 松开，熄灭 D1 */
        }
    }
}
```

- **结果：**按住 K1 时 D1 亮、松开即灭；if 若移出 while(1) 则只在上电/复位时判断一次。



课程截图 · 仅供个人学习 · P7 15:12 —— 开发板全景：按复位键后 LED 排灯 D1 点亮（独立按键/矩阵按键区可见）

例 2（P8 按一下、松手翻转一次） 点按 K1，松手时 D1 亮灭状态取反。

- **思路：**加入软件消抖——按下 Delay(20)、while(P3_1==0); 等松手、再 Delay(20)，最后才 `P2_0=~P2_0`。
- **做法：**

```
#include <regx52.h>   /* 自写示例（非原课程源码） */
#include <intrins.h> /* 延时体内用到 _nop_() */

/* 1ms 空转函数体由 STC-ISP 按 12.000MHz / Y1 生成，此处略写 */
void Delay(unsigned int xms)
{
    while(xms)
    {
        /* 计算器生成的 1ms 空转代码（含 _nop_()，已略） */
        xms--;      /* 后置自减：用完再减 */
    }
}

void main()
{
    while(1)
    {
        if(P3_1 == 0)          /* 检测到 K1 按下 */
        {
            Delay(20);          /* 跳过按下抖动(5~10ms) */
            while(P3_1 == 0);   /* 空语句分号：等松手 */
            Delay(20);          /* 跳过松手抖动 */
            P2_0 = ~P2_0;       /* 松手才翻转 D1 状态 */
```

```
        }
    }
}
```

- **结果：**按一下亮、再按一下灭，点按切换稳定不连跳；传 Delay(500) 可先测 LED 1 秒闪烁验证延时。



对变量自增/移位，再取反写 P2；P2=~LED_number 只读变量、不改 LED_number 本身

图 ③　为什么要用中间变量：P2 上电 0xFF 直接自增会溢出卡死，必须用 unsigned char LED_number 中转后再 `P2=~LED_number`（P9 04:32–08:52）。

例 3（P9 按一次、8 灯二进制递增）　每按一次 K1，亮灯位按二进制加一。

- **思路：**用全局中间变量 LED_number 从 0 数起；消抖模板后 `LED_number++`，再 `P2=~LED_number` 取反送 P2（低电平点亮）。
- **做法：**

```
#include <regx52.h>   /* 自写示例（非原课程源码） */
#include <intrins.h>

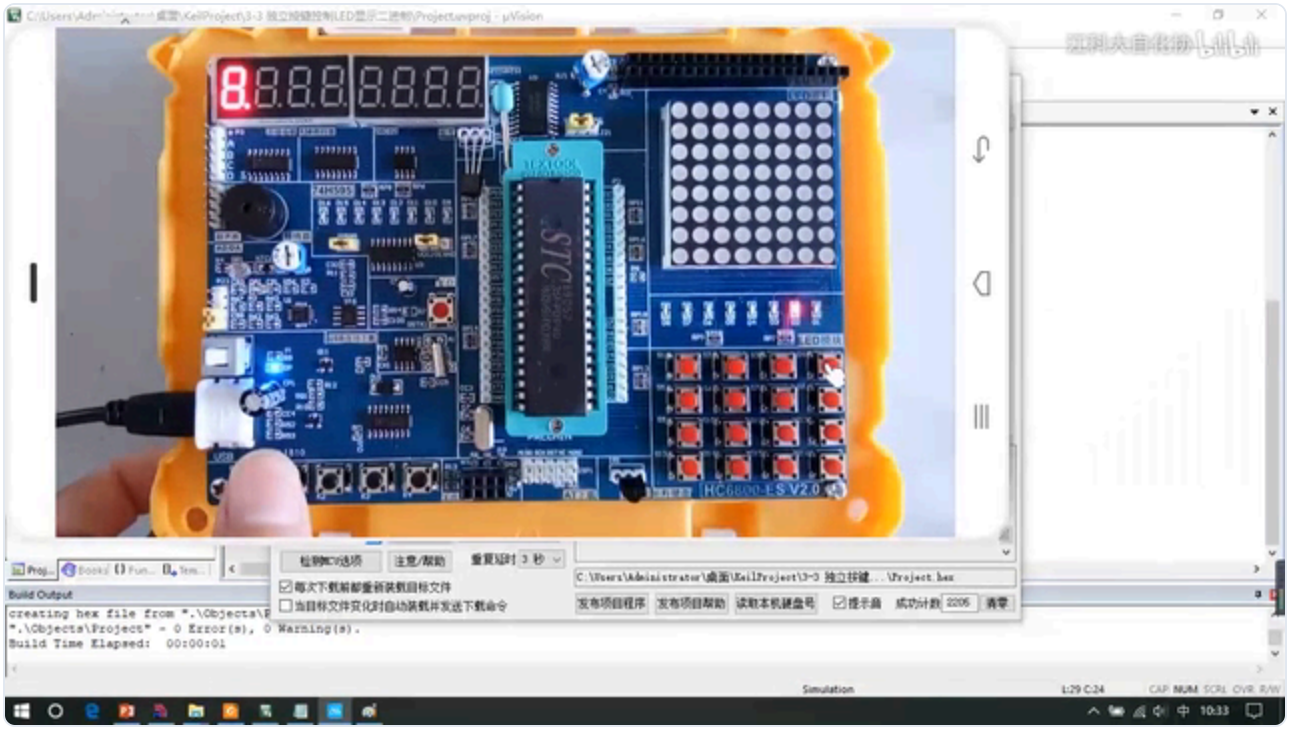
void Delay(unsigned int xms)   /* 同例 2，1ms 空转骨架略 */
{
    while(xms){ /* 1ms 空转(略) */ xms--; }
}

unsigned char LED_number = 0;   /* 中间变量：全局，上电默认 0 */

void main()
{
    while(1)
    {
        if(P3_1 == 0)
        {
            Delay(20);
            while(P3_1 == 0);
            Delay(20);
            LED_number++;          /* 变量自增：0,1,2…255 */
            P2 = ~LED_number;      /* 取反送 P2：亮灯按二进制递增 */
        }
```

```
    }
}
```

- **结果：**LED 从 00000001 亮一位开始逐位加，绕一圈回到全灭再从头；直接写 P2++ 会全亮卡死，这是必须用变量中转的原因。



课程截图 · 仅供个人学习 · P9 09:28 —— 开发板 LED 排灯相邻两颗点亮（二进制计数 00000011=3 演示）

例 4（P10 K1 左移 / K2 右移） K1 按一次亮位左移一位，K2 按一次右移一位，到边界回卷。

- **思路：**用 `0x01 << LED_number` 定位亮位再取反送 P2；K1=P3.1 左移（n++，到 8 归零），K2=P3.0 右移（n--，到 0 跳 7），两个独立 if 平行。
- **做法：**

```
#include <regx52.h>   /* 自写示例（非原课程源码） */
#include <intrins.h>

void Delay(unsigned int xms);  /* 函数原型声明（定义在 main 后时需要，末尾分号） */

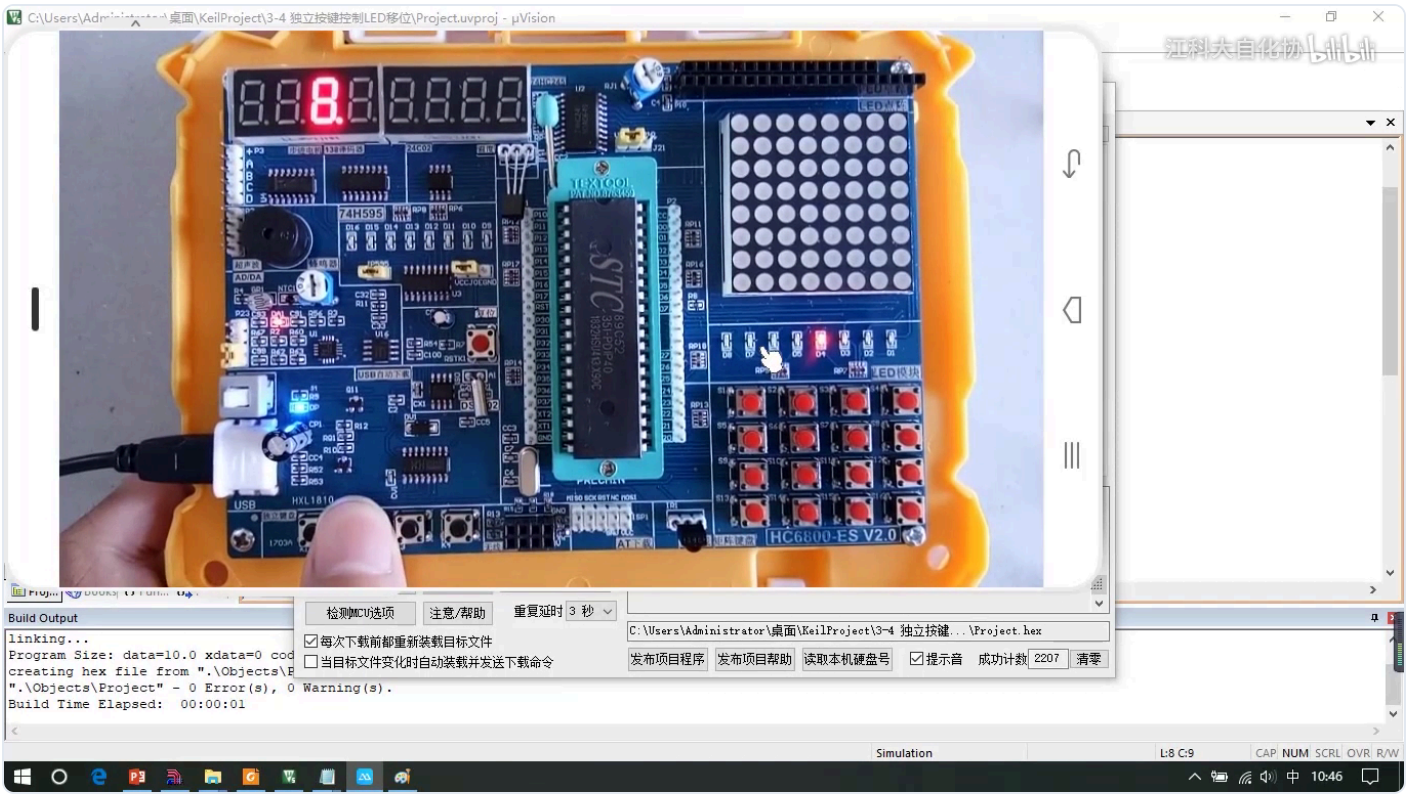
unsigned char LED_number;      /* 全局变量，上电默认 0 */

void main()
{
    P2 = ~0x01;                /* 上电仅 D1 亮，避免按第一下跳到 D2 */
    while(1)
    {
        if(P3_1 == 0)          /* K1：左移 */
        {
            Delay(20); while(P3_1 == 0); Delay(20);
```

```
            LED_number++;
            if(LED_number >= 8) LED_number = 0;   /* 到 8 归零回卷 */
            P2 = ~(0x01 << LED_number);           /* 括号不能省 */
        }
        if(P3_0 == 0)          /* K2：右移（独立 if，不是 else if） */
        {
            Delay(20); while(P3_0 == 0); Delay(20);
            if(LED_number == 0) LED_number = 7;   /* 到 0 跳到最右 */
            else LED_number--;
            P2 = ~(0x01 << LED_number);
        }
    }
}

void Delay(unsigned int xms)   /* 1ms 空转骨架略 */
{
    while(xms){ /* 1ms 空转(略) */ xms--; }
}
```

- **结果：**亮位在 D1–D8 间随 K1/K2 左右流动并循环；少了上电 `P2=~0x01`，第一下就会从 D2 开始。



课程截图 · 仅供个人学习 · P10 11:28 —— 开发板近景：LED 移位点亮效果（独立按键区 K1–K4 可见）

结果速记：例1 按下亮松开灭 = `if(P3_1==0){P2_0=0;}else{P2_0=1;}` 包在 while(1)；例2 点按翻转 = 消抖模板后 `P2_0=~P2_0`；例3 二进制递增 = 中间变量 `LED_number++; P2=~LED_number;`；例4 双向移位 = `P2=~(0x01<<n)`，K1++ 到 8 归零、K2-- 到 0 跳 7。四段共用同一消抖骨架，只有最后动作句不同。

一分钟自测

1. 独立按键一脚接 GND、一脚接 IO；按下时该 IO 被读成高电平还是低电平？程序用什么条件判断"被按下"？
2. 本开发板 K1、K2 分别接哪个 IO？为什么不能按丝印想当然写 K1=P3.0？
3. 要用位名 P2_0、P3_1，应该 include reg52.h 还是 regx52.h？用错会报什么？
4. 机械按键抖动大约多少毫秒？软件消抖模板按顺序写出四步。
5. 为什么不能直接写 `P2++;` 做二进制递增？正确做法是什么？
6. 双向移位里 `P2 = ~(0x01 << LED_number);` 为什么要加括号？上电为什么先写 `P2 = ~0x01;`？

答：

1. 按下接通 GND，IO 被读成**低电平 0**（松开才是内部上拉的高电平 1）；判断条件是 `if(P3_1 == 0)`。
2. 板子反接：**K1=P3.1(RXD)、K2=P3.0(TXD)**；因为 RXD/TXD 丝印画反了，写 P3.0 会按到 K2。
3. 用 **regx52.h**（reg52.h 只有 sfr 没有 sbit）；用错会报"P2_0 未定义的标识符"。
4. 抖动约 **5~10ms**。四步：检测到按下 `Delay(20)` → `while(P3_1==0);` 等松手 → 再 `Delay(20)` → 执行动作。
5. P2 上电默认 **0xFF(全高)**，P2++ 溢出到 0x00、~P2 又回 0xFF，卡死在全亮/全灭两极；正确做法是定义 `unsigned char LED_number` 中转，`LED_number++; P2=~LED_number;`。
6. 括号保证先算 `0x01<<n` 再整体取反，去掉会因运算符优先级出错；上电 `P2=~0x01` 让 D1 默认亮，否则上电全灭、按第一下直接跳到 D2。

本节小结

四集串成一条"**输入 → 输出**"闭环：**读 IO**（按键一脚 GND、按下读 0）→ **抗抖**（Delay(20) 两头消抖 + while 等松手）→ **动作递进**（按下亮灭 → 松手取反 → 中间变量二进制递增 → 0x01<<n 双向移位）。记住三个反复出现的坑：**板子 K1/K2 反接（K1=P3.1）、要用 regx52.h 才有 sbit、别直接对 P2 自增取反而要用 unsigned char 中间变量**。下一章进入 4×4 矩阵键盘。